Chapter 30 worked solutions

Decisions and Embodied Action

Exercises 1–4 trace rewards, tokens, and actions by hand. Exercises 9–11 give sample evaluation designs.

1. Compute returns-to-go

At the last step, the agent receives 4 and the trajectory ends, so the return-to-go at that step is 4. Work backward, adding each step's reward to the sum already found for later steps.

$$\begin{aligned} G_3 &= 4, \\ G_2 &= -1 + 4 = 3, \\ G_1 &= 2 + 3 = 5, \\ G_0 &= 0 + 5 = 5. \end{aligned}$$

The returns in step order are $(5, 5, 3, 4)$. At step 2, the return of 3 includes the penalty of -1. Once that penalty has been received, only the final reward of 4 remains. The recurrence gives the same result, $G_3 = G_2 - r_2 = 3 - (-1) = 4$.

2. Identify the available tokens

Each step contributes its return-to-go, state, and action in that order. At step 2, the action prediction uses the output at s_2.

Step	Tokens in sequence order	Tokens available to the output at s_2
0	G_0, s_0, a_0	None. Step 0 is outside the window.
1	G_1, s_1, a_1	All three are in the window and precede s_2.
2	G_2, s_2, a_2	G_2 and s_2. The causal mask blocks a_2, which comes after s_2.

During training, G_0 is the sum of rewards in the completed episode. For the trajectory in Exercise 1, that sum is 5. At deployment, future rewards are unknown, so the initial return token is a desired return supplied before the first action. After each action, the policy subtracts the observed reward to get the next desired return.

3. Count the two policies

On the original grid, going right from the start $(0, 0)$ immediately enters the trap and earns -1. The log contains three copies of that one-step trajectory. The remaining trajectory starts by going down and then follows right, right, and up to the goal. Its rewards are $(0, 0, 0, 1)$, so its return-to-go at the start is $+1$.

Fit at the start	Up	Right	Down	Left	Chosen action
State only	0	3	1	0	Right
Conditioned on +1	0	0	1	0	Down
Conditioned on −1	0	3	0	0	Right

The state-only fit counts all four starts together. With a desired return of +1, the start-state lookup uses only the successful trajectory. Changing the desired return to −1 selects the start entries from the three failed trajectories, which all supply the same action, right.

4. Read an action bin

The indexing starts at zero, so bin 1 is the second displacement, −1 cm. Bin 4 is the fifth displacement, +2 cm.

The model learns the meaning assigned to each bin during training. If the controller lists the same displacements in reverse order, it decodes bin 1 as +1 cm. The command is valid, yet the end effector moves in the opposite direction from the displacement the model requested.

9. Split entire episodes

Imagine that an agent visits three neighboring cells on its way to the goal. Splitting by step could put the move into the middle cell in training and the move out of it in test. Both steps share the same episode history and later outcome. In a return-conditioned model, their return tokens can even depend on overlapping future rewards. A test window might also contain steps the model already saw during fitting.

Give each trajectory an episode ID, a layout ID, and a starting position. Assign episodes before extracting any training windows. Keep every observation, action, reward, and return-to-go from an episode in the same split. If a trajectory was copied or replayed verbatim, keep the copies together. Verify the grouping after building the windows. Reserve layout E entirely and use one shared fit for both tests. From layouts A–D, reserve starting positions for the familiar-layout test and separate starts for validation. Fit on the remaining episodes and keep both test sets out of tuning.

For **new starts on familiar layouts**, choose layouts represented in training, here A–D. Within each, reserve complete groups of episodes that start at selected positions. For example, put every episode starting at cell $(1, 0)$ on layout A in the test group for that layout. Fit on other starts and use different reserved starts for validation. Check that the test's (layout, start) pairs never appear as training starts. The policy may have seen a test starting cell later in another episode. That fits the question of new starts on a known layout. A claim about unseen states would need a different test.

For **a new layout**, reserve all episodes from one layout, such as layout E, for test. Fit only on layouts A through D and choose settings without any episodes from E or the held-out starts in A–D. No starting position on E enters training, even if a cell has the same coordinates on another layout. Check whether the policy's observations identify the layout or only the coordinates, since the latter can describe different obstacles at the same position.

Report the number of episodes, starts, and layouts in each split. The 100-trajectory total alone does not establish the count per layout or start. Report both tests separately and label cases where a layout has too few different starts for a meaningful familiar-layout test. If possible, repeat the layout holdout with different reserved layouts to see how sensitive the conclusion is to the choice.

10. Compare imitation with closed-loop success

Suppose the correct first move is down from a junction. One policy sometimes goes left into a side alcove that never appeared in demonstrations. The alcove has a way back to the route. Two policies can agree with demonstrations equally often and still differ in what they do from this new state.

Build a simulator test with starts and goals chosen before examining either policy's rollouts. Include ordinary starts and several junctions where a legal off-route move creates a recoverable error. Reset both policies and the simulator to the same state for each start. Keep the observation interface, action rules, reward function, desired return if used, and step limit the same. If the simulator is stochastic, pair its random seeds across the two policies.

Run unperturbed episodes first. Let each action change the simulated world and use the resulting observation as the next input. This closed-loop test exposes the histories the policies actually create. Then run a paired recovery test. After a fixed prefix replayed identically for both policies, execute the same legal off-route action, such as a move into the alcove. Verify that the arrival state is absent from the training demonstrations. Give each policy the actual resulting observation and, if it reads past actions, the action that was actually executed. Update a desired return token using the rewards actually received. Release control back to the policies. The perturbation puts them in the same unfamiliar state and lets us compare recovery directly.

Record whether each run reaches the goal, total reward, number of executed steps, and the ending reason. Distinguish an unsupported action or state, a trap, and a step limit. For unperturbed runs, record how often the agent leaves the familiar route. For perturbed episodes, record whether the agent returns to the route and how many additional steps it takes to reach the goal or terminate. A policy that recovers can achieve a higher goal rate even if it makes the same number of errors on demonstration states. Report unperturbed and recovery results separately so a gain in one does not hide a weakness in the other.

11. Evaluate the control interface

Consider a pick-and-place robot that predicts eight actions from a camera image. After the first action, the object slips sideways. Continuing the original plan can move the gripper toward the old position. Refreshing the image earlier gives the same trained model another chance to locate the object.

Compare two execution policies. The first executes up to eight actions before getting a new image. The second executes up to two. In both cases the model predicts a block of eight. The two-action policy discards the unused suffix, obtains the current image, and predicts a fresh block. Use the same checkpoint, instruction, action encoding, controller, control step duration, and stopping criteria. Record only executed actions in any action history used for the next prediction.

Use paired starting conditions with the same object, robot pose, lighting, camera placement, and target. For example, in a simulator compare both policies on nominal trials and on trials where the object shifts just after the first executed action. Use the same displacement in each matched pair. Keep the disturbance time relative to physical progress the same so one policy does not receive an easier perturbation. Fix the success criterion and trial time and action limits before the comparison.

For success, record whether the robot places the object in the target region before the limit, whether it drops or misses the object, and the final position error. For timing, log frame capture, model start and end, command dispatch, and motion completion on a common clock. Attach the source frame ID to each command. For each position in the block, report observation age as the time between capture and execution. Also report time to completion, model calls, capture-to-first-command latency, and time waiting for a refreshed plan. Refreshing more often may use more inference time even when each action takes the same time to execute.

For disturbed trials, record when the shift happened, when the next observation reflected it, and when the first corrective motion started. Define recovery as regaining a secure grasp after the shift and report eventual placement success separately. Report the fraction that recover and the time and action count from the shift to recovery. Break non-recoveries down by ending reason, including timeout. Comparing these outcomes on the matched starts shows whether more frequent observations trade extra planning time for better correction of errors.